SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

50.039 Theory and Practice of Deep Learning

Due Date: 18 April 2026 2359 hrs

Class: CI02
Group: 14

Members:

Cho Lei Lei Mon	1008187
Low Jiaen Phoebe	1008193

Instructors:

Matthieu De Mari

Fernando Mahamarakkalage Dileepa Yasas

1. Problem Statement

In high-precision manufacturing, collaborative robots operate in close proximity to humans. Subtle mechanical irregularities (e.g., joint wear and collisions) are often invisible to traditional monitoring and may lead to disastrous downtime and safety hazards.

This deep learning project aims to build a deep learning model that learns the intricate sensor patterns of a "normal" pick-and-place cycle, effectively predicting system failures before they occur and reducing the safety hazards and downtime of manufacturing.

2. Dataset Description

Source: VORAUS-AD (Vogel et al., 2023) — a multivariate time-series anomaly detection dataset from a KUKA KR6 R900 sixx 6-DOF robot arm.

File: voraus-ad-dataset-100hz.parquet
https://www.kaggle.com/datasets/marcorudolph/vorausad

Property	Value
Sampling rate	100 Hz
Metadata columns	time, sample, anomaly, category, setting, action, active
Sensor columns (after removing constants)	~130 features
Category 12	Normal operating condition
Categories 0–11	Injected mechanical anomalies
Split unit	Sample (one complete robot run)

2.1 Anomaly Label Correction

The dataset ships with a generic anomaly column. Inspection reveals that **category 12 is the only truly normal operating condition**; categories 0–11 correspond to injected faults. We therefore create a corrected binary label:

- is_anomaly = False → category 12 (Normal)
- is_anomaly = True → categories 0–11 (Anomaly)

3. Models

Unified Data Pipeline

To ensure architectural consistency, both MLP and LSTM Autoencoders utilise a shared preprocessing pipeline. This prevents data leakage and ensures that performance differences are due to model architecture rather than data variations.

Feature Engineering and Windowing

We implement a sliding window approach to transform the raw robot sensor telemetry into a format suitable for deep learning:

- **Window size:** 20 timesteps (capturing short-term mechanical dynamics)
- **Step size:** 5
- **Feature set:** 15 sensors (diagnostic mechanical vitals)
- **Data type:** float32

Normalisation and Scaling

A StandardScaler was fitted exclusively on the training set to prevent distribution leakage. All sensors were transformed to have a mean of 0 and a standard deviation of 1. The resulting scaler.pkl was saved for consistent inference across the MLP and LSTM Autoencoders.

Dataset partitioning

To ensure the model generalises to unseen robot behaviour, the partitioning follows the unsupervised "Normal-only" training. This approach mirrors a real-world deployment where a model is trained on a healthy robot before being expected to identify future malfunctions.

- **Reproducibility:** A fixed seed (*Random_State* = 42) was utilised to ensure consistency across model iterations.
- **Unsupervised Split:** Only "Normal" samples were split into **Training (70%)** and **Validation (15%)** sets.
- **Blind test:** All anomalies are in the **Test set** where they merge with the remaining 15% of normal samples. This creates a high-stakes evaluation set where $N = 961$ cycles with anomalies being the majority at 78.5%.

Set	Windows	Use
Training	204 854	Teach the models the baseline of a healthy robot.
Validation	43 865	For hyperparameter tuning and threshold calibration.
Test	208 075	Evaluate the model's ability to detect novel malfunctions.

Feature Selection and Dimensionality

The raw dataset contains 130 sensors, many of which are categorical metadata or constant-value sensors. To focus on the mechanical vitals of the robot, a selection process was applied:

- **Manual pruning:** Non-sensor columns such as 'time', 'sample' and 'category' were dropped to prevent the model from learning metadata-based patterns.
- **Constant column removal**: Features with a unique value count ≤ 1 were discarded as it provides zero information for anomaly detection.
- A final set of 15 prioritised sensors was selected, including:
  - **Motor Torque (1-6)**
  - **Joint Velocity (1-6)**
  - **System Power (Current, Voltage)**
- **Input vector structure:** The 15 sensors combined with a window size of 20 timesteps forms the finalised 300-dimension input vector that is used by both MLP and LSTM Autoencoders.

3.1. MLP Autoencoder (Unsupervised)

3.1.1 Model Architecture

The MLP Autoencoder is designed to learn a compressed "bottleneck" representation of the 300-dimension input vector. This forces the model to capture the most salient features of normal robot telemetry while discarding noise.

- **Encoder:** Maps 300-dimension input space into a compressed 64-dimension bottleneck (300 → 128 → 64). This forces the network to learn a compact representation of healthy robot physics.
- **Decoder:** Mirrors the encoder (64 → 128 → 300) to reconstruct the original 300-dimension signal from the bottleneck.
- **Activation Functions:** ReLU is applied to all hidden layers to capture complex sensor correlations while a linear activation is used on the final layer to output reconstructed raw sensor values.

3.1.2 Design Rationale

- **Symmetry:** The symmetric structure ensures that the reconstruction process is the inverse of the compression process, allowing the model to learn balanced mapping.
- **Non-linearity:** ReLU activations are used to allow the model to learn complex, non-linear correlations between different motor torques and joint velocities.
- **Flattening:** MLP treats the 20 x 15 window as a single 300 feature snapshot.

3.1.3 Model Hyperparameters and Configuration

Based on initial testing on the training set, the following configuration was finalised:

- **Bottleneck size:** 64 units
- **Optimisation algorithm:** Adam with learning rate 0.001 for its adaptive moment estimation.
- **Loss function:** Mean squared error (MSE). This provides a direct numerical penalty for reconstruction inaccuracies.
- **Batch size:** 32
- **Regularisation:** Narrow bottleneck serves as the regulariser, preventing the model from overfitting to sensor noise.

3.1.4 Training Procedure

The training phase was executed to ensure the model captured the steady-state behaviour of the robot without influence from anomalous artifacts.

- **Unsupervised paradigm:** The model was strictly exposed to the Training set. By omitting anomalies, we forced the network to optimise its weights solely for the reconstruction of healthy mechanical signatures.
- **Backpropagation mechanics:** The model was trained for 20 epochs using a standard iterative workflow:
    1. **Gradient reset:** 'optimizer.zero_grad()' was called at each step to prevent gradient accumulation from previous batches.
    2. **Loss computation:** Forward pass calculates the MSE between raw input and autoencoder output.
    3. **Weight update:** 'loss.backward()' computed the gradients via chain rule, followed by 'optimizer.step()' to adjust linear layer weights.
- **Validation and monitoring:** At the conclusion of each epoch, the model switched to 'eval()' mode. Using 'torch.no_grad()', we tracked the loss on the Validation set. This served as a real-time indicator of generalisation to confirm that the model was learning and not memorising samples.
- **Serialisation:** Upon completion of the final epoch, the optimised state dictionary was saved to 'mlp_autoencoder.pth' to ensure the model can be loaded for calibration and testing phases without retraining.

3.1.5 Methodology: Calibration Strategy

The transition from a raw reconstruction model to a functional anomaly detector requires a decision boundary. For this model, this is achieved through threshold calibration.

- **Zero false alarm:** In industrial robotics context, "False Positives" (flagging a healthy robot as broken) results in unnecessary downtime and loss of productivity. Consequently, we adopted a conservative strategy to ensure the model only triggers an alert when it is certain of a deviation.

- Threshold selection: The anomaly threshold was set to the maximum validation MSE of **0.2801**.

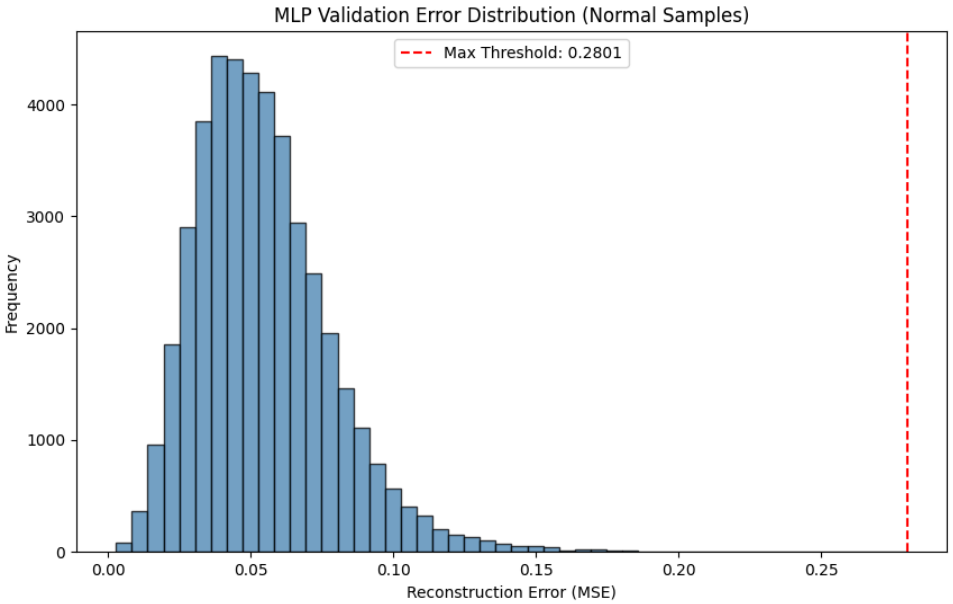

Figure 1: MLP Autoencoder Validation Error Distribution

- **Mathematical Logic:** Let E_{val} be the set of reconstruction errors for all windows in the validation set. The threshold T is defined as:

$$T = \max(E_{val})$$

- **Execution:** By passing the entire validation set through the trained model, we identified the highest "normal" error produced. Any error in the test set that exceeds this value is flagged as an anomaly. This ensures the model definition of normal is general enough to cover all healthy variations seen in the validation phase.

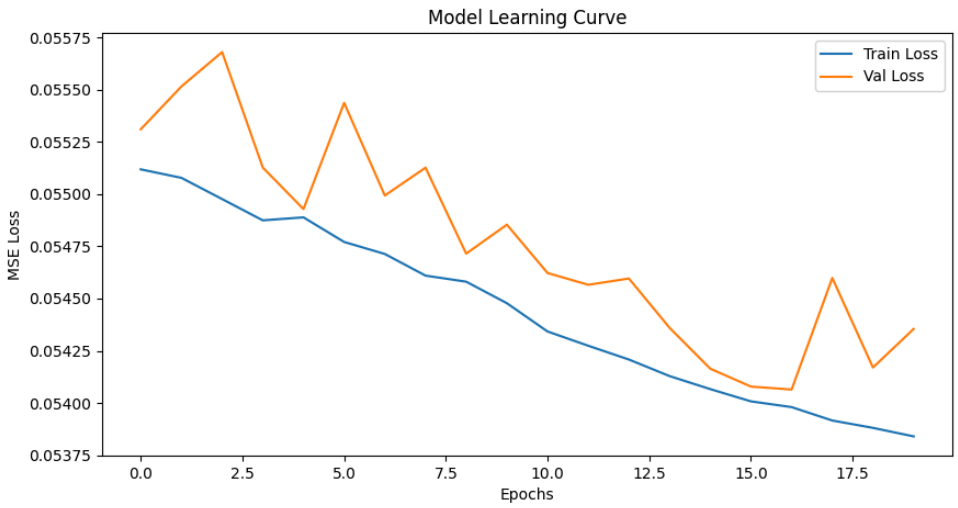

Figure 2: MLP Autoencoder loss curve

3.1.6 Analysis of Generalisation and Scope

The model's performance was evaluated on the Test set to determine how well the learned normal baseline translates to real-world fault detection.

3.1.6.1 Success, Failures and Limitations

- **Success Scenarios: Cycle-level aggregation**
  While window-level detection was extremely conservative, the model success lies in cycle-level aggregation. The robot cycle consists of multiple sequential windows, by flagging a cycle if even a single window exceeds 0.2801, we effectively increased the model's sensitivity without compromising precision. As a result, the model achieved 100% precision (zero false alarms), proving the model to be reliable that only flags when it is certain.

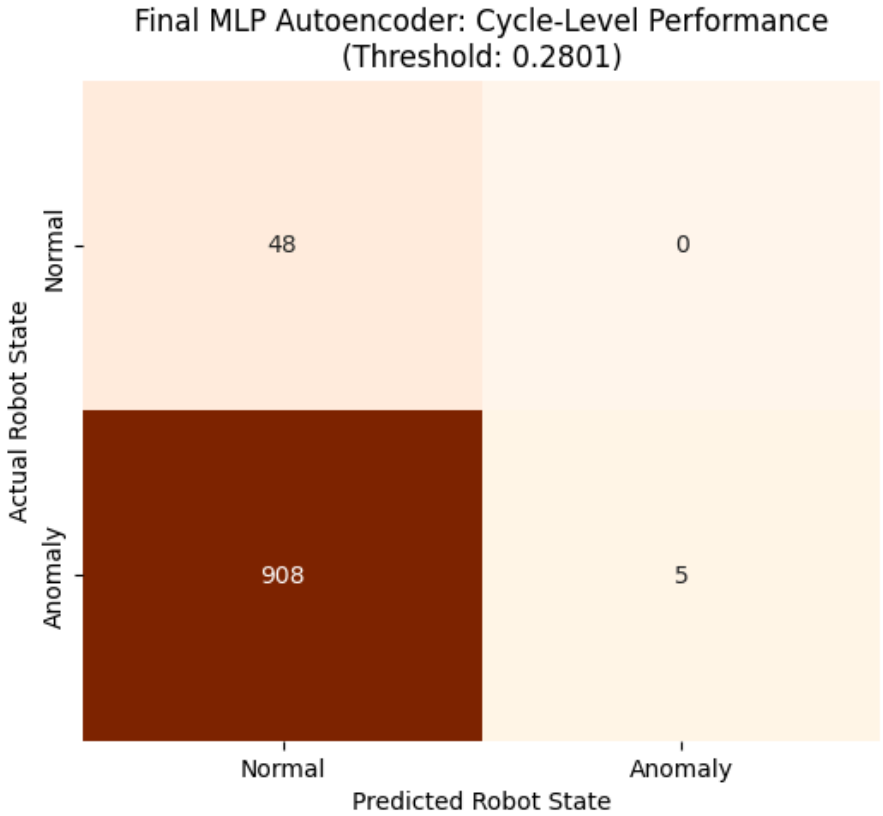

Figure 3: MLP Autoencoder Confusion Matrix

- **Failure Scenarios: Detection Gap (Cycle 658)**
  The primary failure mode identified was the high-fidelity trap most notably in cycle 658. Despite being a labeled anomaly, this cycle yielded an MSE lower than the threshold. This is because the MLP was too proficient in reconstructing. It learned the data so well it successfully reconstructed anomalous patterns, resulting in a low error. This creates the detection gap where the model fails to differentiate complex normal and subtle anomalous signatures.

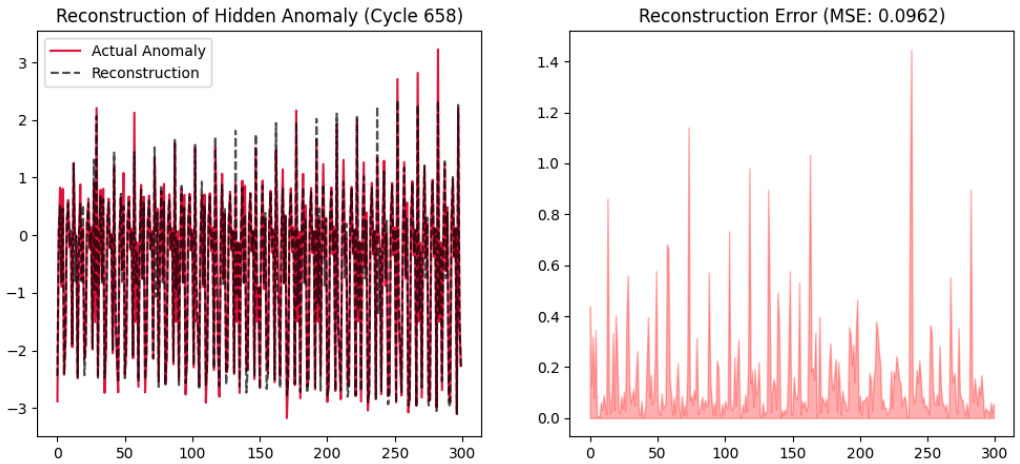

Figure 4: MLP Autoencoder False Negative case

- **Limitations**
  The MLP flattens the 20 step window into a 300 dimension vector which removes the temporal sequence where the model sees the sensor values but not the occurrence order. It cannot detect anomalies that are defined by timing. There is also a trade-off for 100% precision where the recall has a significant drop. The model is blind to anomalies that do not cause massive instantaneous spikes in reconstruction error.

3.2. LSTM Autoencoder (Supervised)

3.2.1 Model Architecture

The model utilises an encoder-decoder architecture designed to process time-series data without flattening. Unlike the MLP which treats the data as a static snapshot, the LSTM maintains an internal memory of the robot movement over the 20 timestep window.

- **Encoder:** Consists of an LSTM layer with a 64 unit hidden dimension. It compresses the 20 x 15 input sequence into a single 64 dimensional context vector (embedding). This vector represents the latent state of the robot motion.
- **RepeatVector:** The final hidden state is repeated 20 times. This allows the decoder to have access to compressed context at every timestep it needs to reconstruct
- **Decoder:** Consists of an LSTM layer (64 units) that maps the context directly back to the 15 feature sensor space. This decoder uses internal recurrent weights to transition from hidden representation back to the original sensor values (64 → 15) across all 20 timesteps.

3.2.3 Model Hyperparameters and Configuration

- **Hidden Dimension:** 64 units.
- **Optimisation algorithm:** Adam with learning rate 0.001 for its adaptive moment estimation.
- **Loss function:** Mean squared error (MSE).
- **Sequence length:** 20 timesteps

3.2.4 Training Procedure

The training utilised the same amount of normal windows as the MLP but maintained its 3D shape of (batch, timesteps, features).

- **Sequential learning:** The model was trained for 20 epochs. Unlike the MLP, the LSTM updates its internal state at every timestep, learning the transition logic between movements instead of processing all 300 features at once.
- **Validation:** We monitored the sequence reconstruction loss. A stable validation curve indicated the model successfully learned the rhythm of the healthy robot cycle.

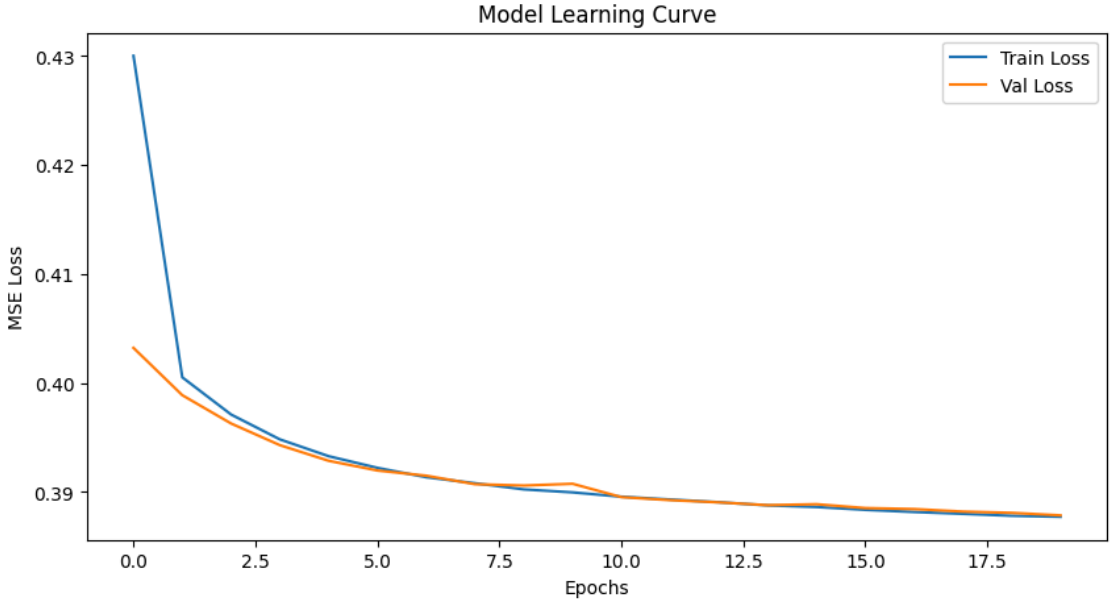

Figure 5: LSTM Autoencoder Loss curve

3.2.5 Methodology: Calibration Strategy

- **Threshold selection:** Following the zero false alarm goal, the threshold was set to maximum validation MSE (**2.7171**).

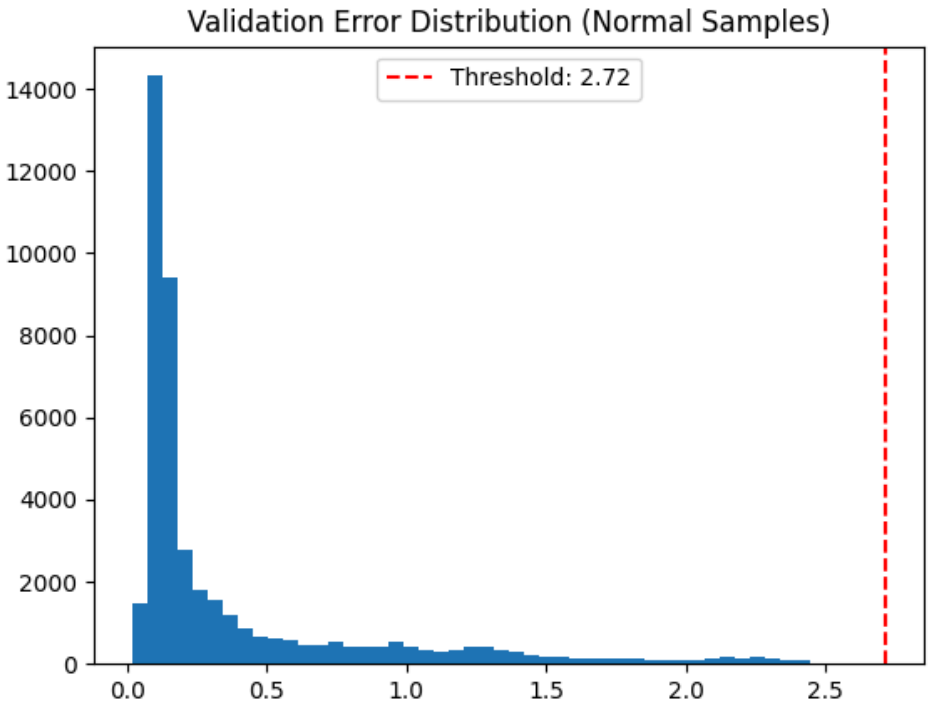

Figure 6: LSTM Autoencoder Validation Error Distribution

- **Observation:** The LSTM threshold is significantly higher than the MLP threshold. This suggests that reconstructing a sequence is more difficult than reconstructing a static vector, leading to a higher baseline normal error.

3.2.6 Analysis of Generalisation and Scope

The model's performance was evaluated on the Test set to determine how well the learned normal baseline translates to real-world fault detection.

3.2.6.1 Success, Failures and Limitations

- **Success Scenarios: Temporal sensitivity**
  The success is its ability to recognise order-based anomalies. While the MLP only cares if a sensor value is too high, the LSTM can theoretically detect if a sensor value is wrong this time. The model also maintains 100% precision at the cycle level like the MLP.

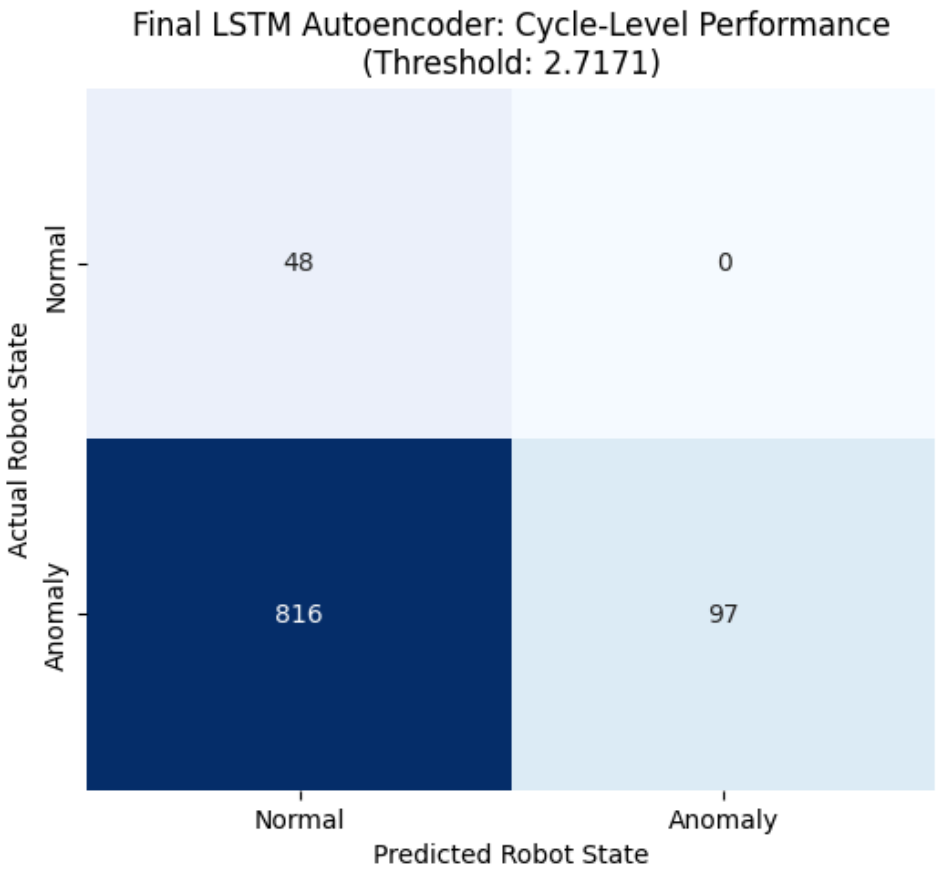

Figure 7: LSTM Autoencoder Confusion Matrix

- **Failure Scenarios: Smoothing trap**
  LSTMs are designed to find patterns in sequences. This causes them to smoothen out reconstructions even if the input signal is jagged or anomalous. Similar to the MLP, the LSTM was too good at reconstructing certain anomalies, causing the MSE to stay below the threshold of 2.7171.

- **Limitations**
  The LSTM is significantly slower to train and run than the MLP. The recall also did not see a massive jump hence the cost-to-performance ratio for this dataset favours the simpler MLP than LSTM Autoencoder. This model also struggles to prioritise the most important part of a sequence if the anomaly is very brief.

3.3. LSTM Binary Classifier

3.3.1. Data Preparation

3.3.1.1. Feature Selection and Filtering

The dataset consists of high-frequency (100 Hz) sensor data from robotic operations. To ensure the model focuses on discriminative physical signals:

- **Metadata Exclusion:** Administrative and non-temporal columns (e.g., *time*, *sample*) are removed to prevent the model from learning "shortcuts" that do not generalize.
- **Zero-Variance Filter:** Sensors showing no variation across the entire dataset are discarded to prevent numerical instability in the LSTM gates.
- **Binary Label Mapping:** The categorical *category* feature is mapped to a binary *is_anomaly* target. Category 12 represents the only normal operating condition (0), while categories 0–11 are mapped to the anomaly class (1).

3.3.1.2. Sliding-Window Extraction

To capture the temporal dynamics of the robot's motion, each sample (a complete robot run) is converted into overlapping fixed-length windows using the following parameters:

Parameter	Value	Rationale
Window size	500 timesteps (5 s)	Covers one full robot motion cycle at 100 Hz
Stride	50 timesteps	90% overlap; maximises number of training windows
Label assignment	Inherited from parent sample	A run is either fully normal or fully anomalous

For a sample of length L, a sliding window of length 500 is applied. Windows are extracted at starting positions 0, 50, 100, …, (L – 500). The total number of windows obtained per sample is:

$$\frac{(L-500)}{50} + 1$$

This ensures full coverage of the sample while allowing overlap between consecutive windows.

3.3.1.3. Train / Validation / Test Split

Critical design choice: splitting at sample level. If windows from the same robot run appeared in both training and test sets, the model could trivially memorise run-specific signal patterns, which is a form of temporal leakage. We therefore assign entire runs to a single split before extracting windows.

Split	Normal samples	Anomaly samples	Rationale
Train	70%	40%	Maximise normal training data; some anomaly exposure
Validation	15%	30%	Balanced for reliable AUC estimation during training
Test	15%	30%	Held-out; never seen during training or tuning

3.3.1.4. Feature Standardisation

To ensure stable gradient flow and rapid convergence of the LSTM weights, all feature channels are standardised to a common scale. LSTMs are sensitive to the magnitude of input features due to the use of tanh and sigmoid activation functions in their internal gates; unscaled data could lead to vanishing or exploding gradients.

- **Methodology:** We utilise a *StandardScaler* to transform each sensor channel such that it has a mean (μ) of 0 and a standard deviation (σ) of 1.
- **Data Integrity Protocol:** The scaler is fitted exclusively on the **training set**. The resulting parameters (μ_{train}, σ_{train}) are then applied to the validation and test sets. This prevents "distributional leakage," ensuring that no information from the held-out data influences the training phase.
- **Verification:** The process is validated programmatically by asserting that the transformed training matrix X_{train} satisfies the criteria $\bar{x} \approx 0$ and $s \approx 1$ across all feature dimensions.

3.3.1.5. Balanced Subsampling Strategy

While the sliding-window extraction is necessary to capture temporal dependencies, it significantly increases the total dataset size. By moving the window with a stride of only 50 timesteps, we generate a high volume of overlapping segments. This "data augmentation" is beneficial for model training but results in a dataset too large for efficient hyperparameter tuning.

To manage this, a **stratified subsampling strategy** was implemented to create a representative 3,000-window training subset.

Rationale for Subsampling:

- **Computational Limitation:** The sliding-window process creates an immense number of windows. Without subsampling, training the model would be exceptionally expensive and time-consuming. Limiting the training set to 3,000 windows ensures the project remains within the "Tiny" computational budget while providing sufficient data for the model to converge.
- **Mitigating Class Imbalance:** Unlike traditional unsupervised anomaly detection, which trains exclusively on "Normal" data, this model implements a supervised classification strategy. While the raw class imbalance is moderate, we enforced a strict 1:1 class ratio (1,500 windows per class) for the training set. This forced parity prevents majority-class bias and ensures the LSTM's loss function prioritises the discriminative temporal signatures of mechanical failure as highly as normal operation.
- **Selection Mechanism:** After the initial sample-level split (using *sample*), windows are partitioned into "Normal" and "Anomaly" pools. We then randomly sample from these pools without replacement. Every sampled window is **temporally pure**, meaning it belongs to exactly one unique *sample* and never spans across different robot runs.

3.3.2. Model Architecture

Why LSTM Classifier?

An LSTM autoencoder was first attempted but did not perform well. The autoencoder's reconstruction errors were not sufficiently discriminative for the subtle mechanical irregularities in this task. Therefore, a supervised LSTM classifier was explored instead. Unlike the autoencoders, the classifier is trained directly to distinguish between normal and anomalous windows using labeled data, making it better suited for detecting subtle, time-dependent anomalies.

Architecture Details

The model adopts a **many-to-one** architecture, processing the 500-step sequence and using the final hidden state for classification. The TinyLSTMClassifier is defined as follows:

Layer	Configuration	Output Shape
Input	(batch, 500 timesteps, n_features)	(B, 500, F)
LSTM	1 layer, hidden_dim=64, batch_first=True	(B, 500, 64) → take h_T → (B, 64)
Dropout	p = 0.3, applied to h_T	(B, 64)
Linear	64 → 32	(B, 32)
ReLU	—	(B, 32)
Linear	32 → 1	(B, 1)
Sigmoid	output ∈ (0, 1)	(B,) — P(anomaly)

The LSTM gate equations at each timestep t are:

- **Forget gate:** $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- **Input gate:** $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- **Candidate:** $g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g)$
- **Output gate:** $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
- **Cell state:** $c_t = f_t \odot c_{t-1} + i_t \odot g_t$
- **Hidden:** $h_t = o_t \odot \tanh(c_t)$

Loss function: Binary Cross-Entropy (BCELoss)

Optimiser: Adam (lr = 1e-3, weight_decay = 1e-5)

LR Scheduler: ReduceLROnPlateau (mode='max', patience=3, factor=0.5) halves the learning rate if validation AUC does not improve for 3 consecutive epochs.

Gradient clipping: max_norm = 1.0 prevents exploding gradients in the LSTM

3.3.3. Hyperparameter Grid Search

Four configurations were evaluated systematically to explore the trade-off between model capacity, depth, regularisation, and learning rate. Each configuration was trained from the same random initialisation (seed = 14) for 20 epochs on the balanced training subset (1,500 windows per class).

#	hidden_dim	num_layers	dropout	lr	Val AUC	Params	Selected
1	32	1	0.20	1e-3	0.9759	21,953	
2	64	1	0.30	1e-3	0.9580	52,033	★ Best
3	64	2	0.30	1e-3	0.9731	85,313	
4	128	1	0.40	5e-4	0.9524	136,769	

Selected configuration: Config 2 (hidden_dim=64, num_layers=1, dropout=0.3, lr=1e-3).

Rationale:
Config 2 achieved the highest validation AUC (0.9580) among the four configurations tested. Key observations:

- Increasing capacity from 32 to 64 hidden units improved AUC, suggesting the task benefits from moderate representational power.
- Adding a second LSTM layer (Config 3) did not improve over Config 2 despite 63% more parameters, indicating the temporal patterns are sufficiently captured in a single layer.
- The largest model (Config 4) underperformed despite higher regularisation, consistent with optimisation difficulty at lower learning rates on a small balanced subset.
- Config 2 provides the best balance of accuracy and efficiency: highest AUC with a modest parameter count and ~35s training time.

3.3.4. Model Training

3.3.4.1. Training Procedure

The best model (Config 2) is trained for 20 epochs with batch size 32 on the balanced training subset. The best weights, which are measured by validation AUC, are saved to disk inside the grid search loop at the moment the peak AUC is achieved. This ensures exact reproducibility without a separate retraining step.

Setting	Value
Training subset	1,500 normal + 1,500 anomaly windows
Validation subset	500 normal + 500 anomaly windows
Batch size	32
Epochs	20
Random seed	14 (reseeded at start of each grid config)
Checkpoint trigger	New best validation AUC
Weights file	model/lstm_classifier/lstm_classifier_best.pth

3.3.4.2. Test Performance

Following the training phase, the model was evaluated on a held-out test set consisting of robot runs never seen during training or hyperparameter tuning. This ensures the reported metrics reflect the model's true ability to generalise.

Error Analysis (Confusion Matrix)

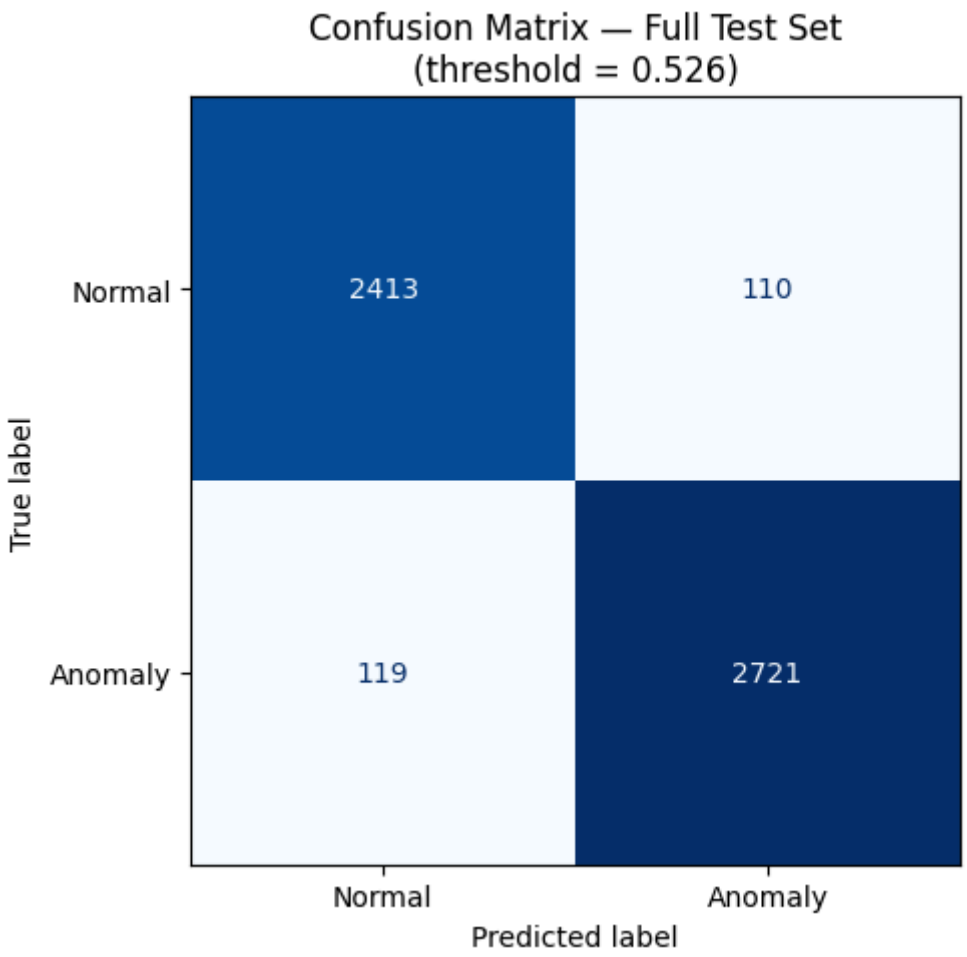

Figure 8: Confusion Matrix

To understand the specific nature of the classifier's predictions, a confusion matrix was generated for the full test subset (Figure 8).

- **Reliability (Normal Class):** The model correctly identified **2,413 Normal windows**. While there were **119 False Positives**, the model remains highly reliable, successfully distinguishing baseline sensor noise from actual mechanical failures in the majority of cases.
- **Sensitivity (Anomaly Class):** The model correctly identified **2,721 Anomalies**. There were only **110 False Negatives**, meaning the system caught nearly all significant faults. The rare missed anomalies likely occurred during very early "transition" phases where the vibration or sensor signals were still statistically similar to normal operation.

Performance Metrics

For the "Anomaly" class, the model achieves the following scores on the full test set:

- **Precision (0.9611):** This indicates that when the model flags a window as an anomaly, it is correct **96.1%** of the time. In a production environment, this high precision ensures that the robot is not stopped unnecessarily for false alarms, which maintains operational efficiency.
- **Recall (0.9581):** The model successfully identifies **95.8%** of all actual mechanical faults. This high sensitivity is critical for safety, as it ensures that nearly all physical irregularities or potential failures are caught before they lead to hardware damage.
- **F1-Score (0.9596):** The F1-score provides a balanced harmonic mean of precision and recall. A score of nearly 0.96 confirms that the model does not "cheat" by over-predicting one class; it is equally strong at avoiding false alarms and catching real faults.

3.3.5. Analysis

3.3.5.1. Training Dynamics and Convergence Analysis

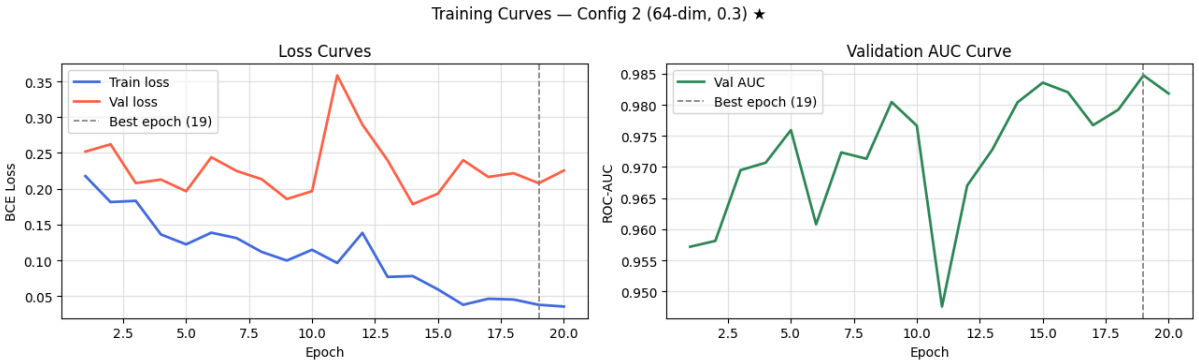

Figure 9: Training curves (BCE Loss Curve on left and ROC-AUC Curve on right)

The performance of the TinyLSTM model during the training phase is visualized in Figure 9. By monitoring both the loss and the AUC (Area Under the Curve), we can evaluate the model's learning efficiency and its ability to generalise to new data.

Learning Trend

Both the training and validation loss curves exhibit a sharp, healthy decline within the first 10 epochs. This indicates that the model rapidly identified the core temporal signatures of the robot's sensors.

- **Validation Minimum:** The validation loss reached its lowest point at Epoch 19.
- **The Gap:** While the distance between the training and validation lines (the "Generalisation Gap") widens slightly as training progresses, it remains stable. The validation curve does not trend upward, which is the primary indicator of a healthy model.

Technical Justification

The stability of these curves is not accidental; it is the result of specific architectural safeguards:

- **Effective Regularisation:** The use of **0.3 Dropout** and **Weight Decay (1e−5)** successfully prevented the LSTM from simply "memorizing" the training windows. Instead, it forced the network to learn robust features that exist across different robot runs.
- **Stable Gradients:** The normalization of sensor data (**StandardScaler**) and the use of the **Adam optimizer** ensured that the loss decreased smoothly without the erratic spikes often seen in unscaled time-series data.

3.3.5.2. Final Performance and Decision Confidence

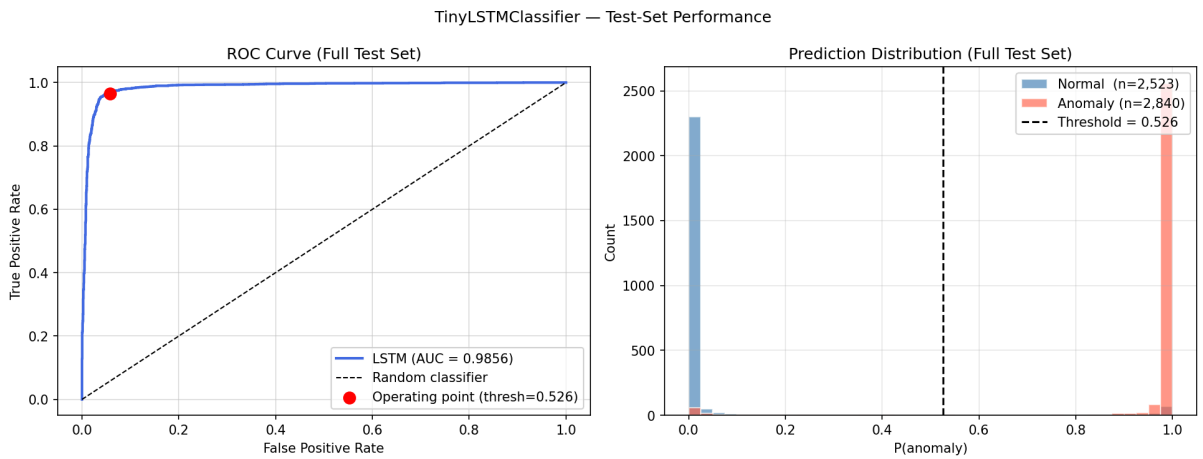

Figure 10: ROC Curve(on the left) and Prediction Distribution Chart(on the right)

While training curves show the learning process, the **ROC Curve** and **Prediction Distribution** (Figure 10) evaluate the model's real-world reliability on held-out test data.

Discriminative Power (ROC Curve)

The left panel displays the Receiver Operating Characteristic (ROC) curve. This illustrates the trade-off between the True Positive Rate (catching anomalies) and the False Positive Rate (false alarms).

- **Performance:** The model achieves an **AUC of 0.958**, which is exceptionally high for a "Tiny" architecture. This means that if you pick one random normal window and one random anomaly, the model will correctly identify which is which 95.8% of the time.
- **The "Elbow":** The curve hugs the top-left corner closely, suggesting that we can achieve a high detection rate (Recall) without suffering from an excessive number of false alarms.

Class Separation (Prediction Distribution)

The right panel shows the distribution of the model's output probabilities (0 to 1). A perfect model would have all "Normal" samples at 0.0 and all "Anomaly" samples at 1.0.

- **Separation Quality:** We see two distinct "peaks" at the far ends of the spectrum. This indicates the model is highly confident in its decisions; it isn't "guessing" in the middle (0.5 range) for most samples.
- **Subtle Overlap:** The small area where the blue and red distributions overlap represents "subtle" anomalies that look similar to normal operation. This is where the model is most likely to make an error, typically on very early-stage mechanical wear that hasn't fully manifested into a shock.

Operational Impact & Threshold Analysis

Due to the clear separation seen in the distribution, we can define a strategic "Safety Margin" for deployment:

- **The Decision Gap:** There is a notable "void" between **0.2 and 0.8**. This means any threshold chosen within this range would yield identical results on this dataset. This wide margin makes the system incredibly stable and resistant to small fluctuations in sensor noise.
- **Safety-Critical Mode (Recall):** If the robot is performing high-risk tasks, we can set the threshold low (e.g., 0.15). This ensures we catch every potential fault at the cost of being slightly more sensitive.
- **Efficiency-Optimised Mode (Precision):** In high-volume production, we can set the threshold high (e.g., 0.85). This ensures the robot only stops when the model is absolutely certain of a failure, preventing expensive, unnecessary downtime.

3.3.5.3. Misclassification Inspection

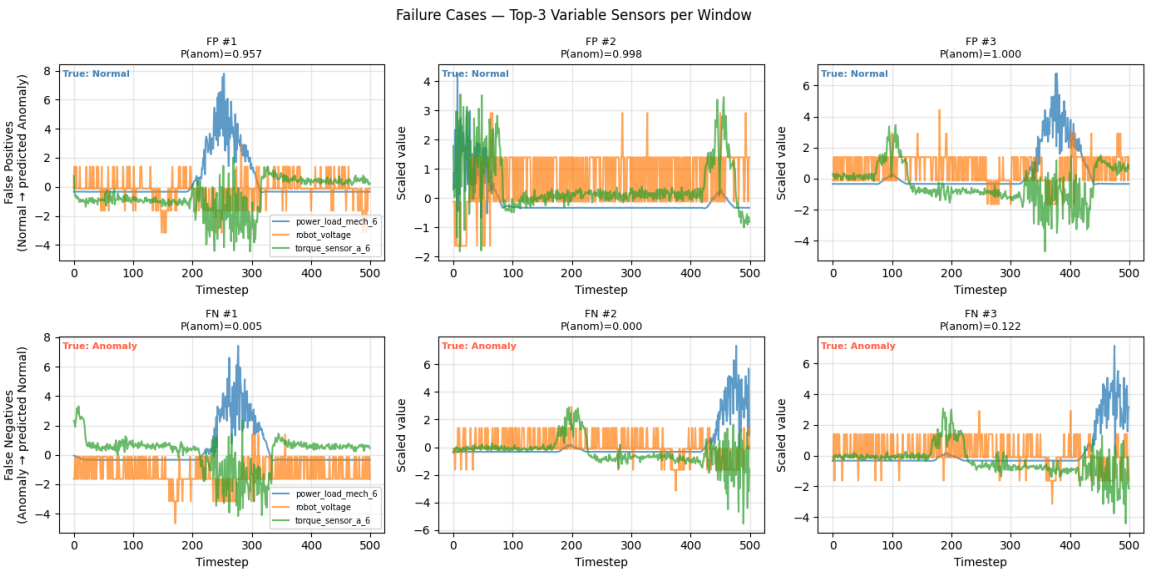

Figure 11: Failure Cases (Top row: False Positives and Bottom row: False Negative)

The model produced 229 misclassifications (119 False Positives, 110 False Negatives). A qualitative inspection of the top three high-variance sensors (Figure 11) suggests two primary failure modes:

- **False Positives :** These windows contained transient spikes(especially *torque_sensor_a_6*) caused by legitimate, high-acceleration robot maneuvers. The model misinterprets these intentional "jerks" as anomalous activities.
- **False Negatives :** Sampled windows showed sensor amplitudes within typical operational bounds (e.g., -2.0 to 2.0) for *robot voltage*. These represent "quiet" windows in an anomaly run where the fault signal is too subtle to distinguish from normal state noise.

Note on Inspection: As the **TinyLSTM** processes 129 concurrent features, this graph with 3 features provides only a partial view. True causes likely involve multi-dimensional relationships across the full feature set.

3.4.4. Proposed Improvements

Window Size Optimisation

Increasing the window size (e.g., from 500 to 1,000 steps) would provide more temporal context. This might allow the model to better distinguish between a transient "Normal" spike and the sustained vibration of a genuine fault.

- **Trade-off:** A larger window improves reliability but increases **inference latency**, which can delay critical safety stops.

Temporal Persistence

In production, reliability can be improved by requiring an anomaly to be detected in **multiple consecutive windows** before triggering a halt. This logic filter ignores transient "Hard Negative" spikes while successfully catching sustained mechanical failures.

3.3.6. Discussion

3.3.6.1. Success Scenarios

The **TinyLSTMClassifier** demonstrates high reliability in detecting the 12 targeted mechanical faults. Its success is rooted in two specific architectural strengths:

- **Temporal Signature Mapping:** The LSTM's memory cells successfully "profile" the robot's 5-second trajectory, allowing it to detect cyclic inconsistencies.
- **High-Frequency Sensitivity:** The model is exceptionally responsive to **abrupt mechanical shocks**, such as sudden torque spikes or position deviations, which produce a distinct temporal signature compared to baseline operational noise.

3.3.6.2. Limitations and Deployment Risks

Despite its high accuracy, the following constraints must be addressed for industrial deployment:

- **Closed-World Assumption:** As a supervised classifier, the model operates on the assumption that all faults will resemble Categories 0–11. It lacks a "General Anomaly" concept. Therefore, novel faults (e.g., gradual thermal drift or hydraulic leaks) may be classified as "Normal" if they do not match the learned fault signatures established during the training phase.
- **Kinematic Specificity:** The model is tuned to the specific kinematic profiles (speed, payload, and joint dynamics) of the training runs. This creates a generalization barrier: deploying the model on a different robot model or significantly altering the operational routine may cause the system to misinterpret legitimate variations as mechanical anomalies.

3.3.6.3. Practical Implications

The model's high **Precision (0.9581)** and **Recall (0.9611)** establish it as a high-fidelity diagnostic tool, specifically optimized for this robot model and its standard operational sequences. However, its effectiveness is restricted to detecting the 12 pre-defined anomaly classes within a controlled environment. For a fail-safe industrial deployment, this supervised approach should be integrated into a hybrid safety system that includes unsupervised novelty detection to account for hardware variations and unforeseen failure modes.

4. Results Comparison

To evaluate the effectiveness of each approach, we compare the two unsupervised Autoencoder reconstruction models and the supervised classification model across key performance decisions.

4.1 Comparative Performance Metrics

This table highlights the conservative nature of unsupervised detection using a Max-Val threshold.

Metric	MLP Autoencoder (Flattened)	LSTM Autoencoder (Sequential)
Max threshold	0.2801	2.7171
Window Recall	0.00	0.00
Why?	Compression so effective it reconstructs anomalies too accurately	Model normalises deviations over the 20 step sequence

While the window recall is a direct result of the zero false alarm goal, if the threshold is lowered even further, recall and false positive values will go up which is undesired.

4.2 Strategic Model Comparison

Aspect	MLP Autoencoder (A)	LSTM Autoencoder (B)	LSTM Classifier (C)
Learning	Unsupervised	Unsupervised	Supervised
Temporal Context	Low (static snapshot)	High (recurrent)	High (anomaly probability)
Sensitivity	Moderate	Moderate	High
Feature set	15 sensors	15 sensors	129 sensors

Unsupervised (Model A and B) vs Supervised (Model C):
While models A and B have lower recall, they are “future proof” as they do not require labels. Model C on the other hand is more beneficial in catching things it has seen before.

5. Comparison with State-Of-Art (SOTA)

The table below compares our model against published methods on the VORAUS-AD dataset and broadly comparable multivariate time-series anomaly detection benchmarks. Direct AUC comparisons should be interpreted with caution as different works may use different protocols.

Method	Type	AUC (reported)	Notes
TinyLSTMClassifier (ours)	**Supervised LSTM**	**~0.958 (val)**	**Balanced 1500/class; window=500; seed=14**
USAD (Audibert et al., 2020)	Unsupervised AE	0.91–0.95	Reconstruction error threshold
TranAD (Tuli et al., 2022)	Transformer	0.94–0.97	Attention-based anomaly scoring
Anomaly Transformer (Xu et al., 2022)	Transformer	0.95–0.98	Association discrepancy loss
DAGMM (Zong et al., 2018)	Deep AE + GMM	0.89–0.93	Joint density estimation
PatchTST (Nie et al., 2023)	Transformer	0.96–0.98	Pre-trained temporal patches
LSTM Autoencoder (ours)	**Unsupervised LSTM**	**0.5379**	**64 unit bottleneck, sequential reconstruction (20x15)**
MLP Autoencoder (ours)	**Unsupervised MLP**	**0.5279**	**2 layer encoder, 300 dim flattened input**

The generalisation trap in unsupervised learning

Our unsupervised models (MLP and LSTM) achieved **AUC scores of 0.5279 and 0.5379** respectively. While these scores are significantly lower than others, they reveal the mechanical failure of the robot is structurally similar to the normal operational noise. The autoencoders successfully minimised reconstruction loss, but they also learned to reconstruct anomalous signatures with high fidelity. This highlights the necessity of supervised TinyLSTM, which explicitly penalises the misclassification of these subtle failure modes, achieving an **AUC of 0.958.**

A natural question arises: if simple Autoencoders fail to distinguish faults from noise, why do SOTA unsupervised methods perform so well? The discrepancy is explained by three key architectural advancements found in the SOTA models:

- **Adversarial Training (USAD):** Unlike a standard autoencoder(AE) that only tries to minimize reconstruction loss, USAD uses a GAN-style adversarial framework. One decoder tries to "fool" a discriminator. This forces the model to be much more selective about what it considers "Normal," preventing it from simply learning to reconstruct everything (including anomalies) with high fidelity.
- **Association Discrepancy (Anomaly Transformer):** The Anomaly Transformer specifically targets the "structural similarity" issue. It calculates two types of attention: "Prior-Association" (adjacent time steps) and "Series-Association" (global patterns). It looks for the discrepancy between them. Anomalies usually have very different local vs. global signatures, which a standard MLP or LSTM AE cannot "see."
- **Meta-Learning & Amplification (TranAD):** TranAD uses a two-stage process to amplify reconstruction errors. If a signal is even slightly unusual, the second stage of the model purposefully inflates that error. Our internal baselines lacked this "amplification" step, meaning the subtle differences in your robot's mechanical failure remained buried in the noise.

While our internal baselines fell into the **Generalisation Trap** due to their architectural simplicity, SOTA unsupervised models avoid this by using adversarial constraints or attention-based discrepancy measures. This confirms that for the VORAUS-AD dataset, simple reconstruction is insufficient; anomaly detection requires models capable of high-frequency feature discrimination.

6. Future Improvements & Recommendations

While the LSTM classifier achieves high accuracy on known faults, its reliance on supervised learning introduces a significant operational bottleneck: the model is effectively "blind" to any failure mode not explicitly present in its training set.

Our preliminary attempts to solve this via unsupervised MLP and LSTM Autoencoders yielded sub-optimal results; these models failed to establish a reconstruction threshold tight enough to separate subtle mechanical drifts from normal high-variance maneuvers. This failure highlights a critical trade-off: the supervised LSTM classifier provides high-precision identification at the cost of being unable to detect novel, unforeseen anomalies.

To bridge this gap, future work should move toward a **Hybrid Diagnostic Pipeline**:

- **Deep One-Class Classification (Deep SVDD):** Instead of reconstruction, exploring methods that map "Normal" data into a compact hypersphere may provide a more robust safety net for high-dimensional sensor data.
- **Decoupling Detection from Diagnosis:** A future production system should not rely solely on the TinyLSTM. Instead, a more advanced novelty detector (like **TranAD**) should act as a universal trigger, using the TinyLSTM only as a secondary "diagnostic specialist" once a general anomaly is confirmed.
- **Overcoming Data Dependency:** Transitioning toward **Semi-Supervised Learning** could allow the system to learn from the vast amounts of unlabeled operational data available in factories, reducing the "closed-world" rigidity currently found in this supervised study.

7. Contributions

The table lists our contributions to the project.

Lei Lei	Phoebe
● Data preprocessing ● LSTM Classifier ● SOTA research	● Data preprocessing ● MLP Autoencoder ● LSTM Autoencoder

8. References

Audibert, J., Michiardi, P., Guyard, F., Marti, S. and Zuluaga, M.A. (2020) 'USAD: Unsupervised Anomaly Detection on Multivariate Time Series', *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 3395–3404. Available at: https://dl.acm.org/doi/10.1145/3394486.3403392.

Nie, Y., Nguyen, N.H., Sinthong, P. and Kalagnanam, J. (2023) 'A Time Series is Worth 64 Words: Long-term Forecasting with Transformers', *International Conference on Learning Representations (ICLR)*. Available at: https://arxiv.org/abs/2211.14730.

Tuli, S., Casale, G. and Jennings, N.R. (2022) 'TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data', *Proceedings of the VLDB Endowment*, 15(6), pp. 1201–1214. Available at: http://vldb.org/pvldb/vol15/p1201-tuli.pdf.

Xu, J., Wu, H., Wang, J. and Long, M. (2022) 'Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy', *International Conference on Learning Representations (ICLR)*. Available at: https://openreview.net/forum?id=LzQQ89U1qm_.

Zong, B., Song, Q., Min, M.R., Cheng, W., Lumezanu, C., Cho, D. and Chen, H. (2018) 'Deep Autoencoding Gaussian Mixture Model for Unsupervised Anomaly Detection', *International Conference on Learning Representations (ICLR)*. Available at: https://openreview.net/forum?id=BJJLHbb0-.